

This week, you'll learn **Kubernetes & Container Orchestration**. Kubernetes helps DevOps teams run, scale, update, and manage containerized applications reliably across environments.

Topic:- Kubernetes & Container Orchestration

- Kubernetes Fundamentals
- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Namespaces
- kubectl
- Helm — Introduction
- Ingress — Introduction

*Learning Approach

Focus on understanding **how Kubernetes manages containers and keeps applications running**. Instead of only running commands, understand how Pods, Deployments, and Services work together.

Keep these points in mind:

- ✓ Understand the role of each Kubernetes object.
- ✓ Learn the difference between a Pod, Deployment, and Service.
- ✓ Practice using kubectl to inspect and troubleshoot resources.
- ✓ Understand how ConfigMaps and Secrets separate configuration from application code.
- ✓ Use AI suggestions only after understanding and testing them.

***Complete the following practical exercises:**

- Install **Minikube or Kind**.
- Start a local Kubernetes cluster.
- Create and manage Pods.
- Create a Deployment for a sample application.
- Scale the Deployment to multiple replicas.
- Create a Service to expose the application.
- Create a ConfigMap.
- Create a Secret.
- Create a Namespace.
- Practice important kubectl commands.
- Explore basic Helm concepts.
- Explore basic Ingress concepts.

Task Submission

- Kubernetes YAML files
- Screenshot of Running Cluster
- Screenshot of Pods
- Screenshot of Deployment
- Screenshot of Service
- Screenshot of ConfigMap/Secret
- Screenshot of kubectl commands

Task Submission Deadline: Wednesday (EOD)

Scenario

Imagine you are a **DevOps Engineer** responsible for deploying a web application for a development team. The application should remain available even when additional replicas are required.

Use your Kubernetes environment and complete the following:

- Deploy the application using a **Deployment**.
- Run at least **3 replicas**.
- Expose the application using a **Service**.
- Scale the application up and down.
- Perform a **rolling update** by changing the application version.
- Check the status of the rollout.
- Roll back the Deployment to the previous version.
- Use a separate Namespace for the application.
- Draw the deployment flow:

User → Service → Deployment → Pods → Container

Activity Submission

- ✓ Kubernetes Architecture Diagram
- ✓ Screenshot of 3 Running Replicas
- ✓ Screenshot of Scaling
- ✓ Screenshot of Rolling Update
- ✓ Screenshot of Rollback
- ✓ Screenshot of Application Access
- ✓ Short explanation of what happened during scaling and rollback

Activity Submission Deadline: Friday (EOD)

Use an AI Assistant such as **ChatGPT, Amazon Q, GitHub Copilot, Gemini, Claude, or Microsoft Copilot** as a **Kubernetes Troubleshooting Assistant**.

Explore how AI can help you:

- Explain Kubernetes errors and failed Pods.
- Analyze `kubectl describe` and `kubectl logs` output.
- Suggest possible causes for a failed Deployment.
- Generate Kubernetes YAML based on your requirements.
- Review your Kubernetes manifests for configuration issues.
- Explain Service connectivity problems.
- Suggest troubleshooting steps for `CrashLoopBackOff` or `Pending` Pods.
- Compare your troubleshooting approach with the AI's suggestion.

No separate AI submission is required.

The goal is to learn how AI can assist with Kubernetes deployment, troubleshooting, configuration, and day-to-day DevOps operations.

***DevOps Pro Tip**

When a Kubernetes application fails, don't immediately recreate everything. Start with **`kubectl get`**, **`kubectl describe`**, and **`kubectl logs`** to understand what is actually happening.

Understand how Kubernetes manages **Pods, Deployments, Services, and configuration** rather than simply executing YAML files. Strong Kubernetes fundamentals will prepare you for cloud-based deployments, CI/CD, scaling, and production environments.

***If you face any genuine issues, feel free to reach out before the submission deadline.**